
Aligned Training: A Parameter-Free Method to Improve Feature Quality and Stability of Sparse Autoencoders (SAE)

Michał Brzozowski^{1,†}
Neo Christopher Chung^{1,2}

¹Samsung AI Center, Warsaw, Poland ²University of Warsaw, Poland

[†]Corresponding author: m.brzozowsk3@samsung.com

Abstract

Sparse autoencoders (SAEs) are one of the main methods to interpret the inner workings of deep neural networks (DNNs), decomposing activations into higher-dimensional features. However, they exhibit critical shortcomings where a large fraction of features are never activated and are unstable. Despite variants of SAEs that attempt to mitigate these issues, they require additional data, resampling, or training. We propose the **aligned training**, a parameter-free reparameterization of SAEs that simultaneously improves reconstruction quality, eliminates dead features, and significantly enhances stability across training seeds. Our approach is motivated by an overlooked observation that SAE feature quality, measured by the inner product between encoder and decoder directions (which we call the **alignment score**), follows a bimodal distribution across all modern architectures. The proposed aligned training enforces a geometric constraint between the encoder and decoder such that their inner product equals one for every feature, which removes a source of degeneracy in the SAE training without adding any hyperparameters. Across multiple models, dictionary sizes, and sparsity levels, the aligned training shows Pareto improvements on the SAE Bench benchmarks. Beyond improving dead features, stability and reconstruction, our method readily integrates with techniques in mechanistic interpretability such as Top/BatchTop-K architectures and p-Annealing. Overall, the aligned training substantially improves feature quality and stability of SAE without computational complexity or cost.

1 Introduction

Deep neural networks (DNN) have made remarkable strides in advancing natural language understanding and generation. Nonetheless, the inner workings of DNNs remain opaque, which limits their development and application in high-risk settings. Mechanistic interpretability has been developed to address this challenge. The superposition hypothesis suggests that a substantially larger number of independent concepts are disentangled in a lower-dimensional activation space of a DNN [10]. Dictionary learning attempts to recover these concepts by estimating a sparse overcomplete basis.

Sparse autoencoders (SAEs) are among the most popular tools in mechanistic interpretability. Applied to the residual stream of a transformer with sparsity constraints, they decompose activations into interpretable features [15, 5]. Despite the popularity and architectural progress [13, 24, 25], two problems persist across all variants of SAE: a significant fraction of features are *dead* (never activating on any input) [5, 13, 16], and features learned across independent training runs are often *inconsistent* [23, 30].

The root cause is an overlooked degeneracy. The inner product between each feature’s encoder row and decoder column — which we call the **alignment score** — follows a bimodal distribution across all modern SAE architectures (see Appendix C): many features are well-aligned, but a substantial fraction are nearly orthogonal artifacts with alignment scores near zero. Well-aligned features ($a_i \approx 1$) correlate strongly with existing quality proxies such as MCS and autointerpretability, while poorly aligned features correspond to uninterpretable noise. Standard training leaves this degree of freedom entirely unconstrained.

To address this, we introduce **aligned training**: a parameter-free reparameterization that enforces $W_{i,\cdot}^{enc} \cdot W_{\cdot,i}^{dec} = 1$ for every feature i by construction. This single geometric constraint, motivated by a projection onto an affine hyperplane (Section 3.2), simultaneously addresses both failure modes and improves reconstruction quality — without any auxiliary losses, resampling, or additional hyperparameters.

Through extensive experiments, aligned training is shown to (i) achieve Pareto improvements in reconstruction quality, (ii) reduce dead features to near zero, and (iii) significantly improve feature consistency across seeds — on SAE Bench benchmarks across multiple models, dictionary sizes, sparsity penalties, and activation functions.

2 Related Work

2.1 Sparse Autoencoders

Let $\mathbb{R}^n$ represent the activation space of a deep learning model. The goal of a SAE is to decompose a data point’s activation into a sparse linear combination of features from a dictionary of size m . Such features have been shown to be more interpretable than the neuron basis [15, 5].

The encoder and decoder are:

$$\mathbf{f}(\mathbf{x}) := \text{ReLU}(W^{enc}\mathbf{x} + \mathbf{b}^{enc}), \quad \hat{\mathbf{x}} := W^{dec}\mathbf{f}(\mathbf{x}) + \mathbf{b}^{dec},$$

where $W^{enc} \in \mathbb{R}^{m \times n}$, $W^{dec} \in \mathbb{R}^{n \times m}$. The training loss is

$$\mathcal{L} = \underbrace{\|\hat{\mathbf{x}} - \mathbf{x}\|^2}_{\mathcal{L}_R} + \lambda \underbrace{\|\mathbf{f}(\mathbf{x})\|_1}_{\mathcal{L}_P}.$$

An important structural property is homogeneity: multiplying the encoder and dividing the decoder by any scalar leaves the reconstruction unchanged while making $\mathcal{L}_P$ arbitrarily small. Two approaches have been proposed to handle this degeneracy.

Normalization [5, 15] projects decoder columns to unit norm after each gradient step, requiring careful synchronization with the Adam optimizer.

Reformulation [7] replaces $\mathcal{L}_P$ with the decoder-norm-weighted penalty:

$$\mathcal{L}_P = \sum_i \mathbf{f}_i(\mathbf{x}) \|W_{\cdot,i}^{dec}\|_2.$$

This is invariant to the homogeneous rescaling and requires no constrained optimizer. We adopt this formulation throughout. Understanding this symmetry is essential: in Section 3.2 we exploit it to derive aligned training.

2.2 SAE Architectures

Since the original ReLU SAE [15, 5], several improvements have been proposed: Gated SAEs [24], TopK SAEs [13], JumpReLU SAEs [25], and BatchTopK [6]. We primarily analyze ReLU SAEs, but demonstrate that aligned training is architecture-agnostic by showing improvements for TopK and BatchTopK variants.

2.3 Feature Quality, Dead Features, and Stability

A common thread runs through three persistent problems in SAE training: the optimization landscape is underconstrained, and gradient descent can exploit the resulting degeneracy in ways that hurt interpretability.

Feature quality is most commonly measured by maximal cosine similarity (MCS) [29], which trains two SAEs and checks which features are rediscovered by both; such features are considered “universal.” MCS scores follow a bimodal distribution [9, 14] and correlate positively with monosemanticity [3, 26, 8]. However, MCS is expensive to compute, requiring an additional larger dictionary.

Dead features (features that never activate) are a direct symptom of the same degeneracy: zeroing a feature is an easy way to reduce the sparsity penalty at no reconstruction cost. Existing remedies (resampling [5], ghost gradients [16], auxiliary losses [13]) treat the symptom after the fact, each adding hyperparameters or non-differentiable operations. Marks et al. [21] take a different approach, training two SAEs in parallel with a similarity penalty to encourage consistent feature learning, at the cost of doubling the training budget.

Feature consistency is a complementary concern: SAEs trained on the same data with different random seeds often converge to qualitatively different dictionaries [23, 30], undermining reproducibility. This too reflects the underconstrained landscape: multiple basins of equal loss exist, and different seeds fall into different ones.

Aligned training addresses all three problems at the source by constraining the optimization manifold directly, rather than patching each symptom individually.

3 Proposed Methods

3.1 Alignment Score

According to the **linear representation hypothesis**, deep learning models represent concepts as directions in activation space. Each SAE feature i corresponds to the encoder row $W_{i,\cdot}^{enc}$ and the decoder column $W_{\cdot,i}^{dec}$. It is not immediately clear which is the “true” feature direction; in practice the encoder is used for concept detection and the decoder for model steering [32]. Intuitively these two directions should agree, and tied-weight SAEs [15] enforce this as a hard constraint.

We introduce a softer proxy: the **alignment score**

$$a_i = W_{i,\cdot}^{enc} \cdot W_{\cdot,i}^{dec}, \quad (1)$$

the i -th diagonal of $W^{enc}W^{dec}$. We use the inner product rather than cosine similarity because a_i is invariant under the homogeneous rescaling of Section 2.1 (multiplying the encoder and dividing the decoder by any scalar leaves a_i unchanged), matching the symmetry of the training objective. Cosine similarity, being separately scale-invariant in each argument, would not capture this structure.

We conjecture that features with $a_i \approx 0$ or $a_i < 0$ are uninterpretable artifacts of training. This is corroborated empirically: the alignment score follows a bimodal distribution across all modern SAE architectures, strongly correlates with MCS (Pearson $r = 0.65$), and is positively correlated with autointerpretability (Pearson $r = 0.32$). Full characterization is in Appendix C.

Toy model. Consider the minimal setting: $n = 2$, $m = 1$, no biases, a single training point $\mathbf{x} \in \mathbb{R}^2$, and no sparsity penalty. Perfect reconstruction $\hat{\mathbf{x}} = \mathbf{x}$ requires

$$\mathbf{x} = \text{ReLU}(w^{enc} \cdot \mathbf{x}) w^{dec}.$$

Since $\mathbf{x}$ and w^{dec} must be parallel, writing $\mathbf{x} = \alpha w^{dec}$ and substituting yields $w^{enc} \cdot w^{dec} = 1$: perfect reconstruction forces the alignment score to one.

3.2 Aligned Training

The alignment score motivates a direct fix: if well-aligned features ($a_i = 1$) are the good ones, we can reparameterize the encoder to enforce this for every feature by construction. Geometrically, the constraint $W_{i,\cdot}^{enc} \cdot W_{\cdot,i}^{dec} = 1$ defines an *affine hyperplane* in the space of encoder rows, with normal vector $W_{\cdot,i}^{dec}$. We parameterize the encoder by projecting a free parameter onto this hyperplane.

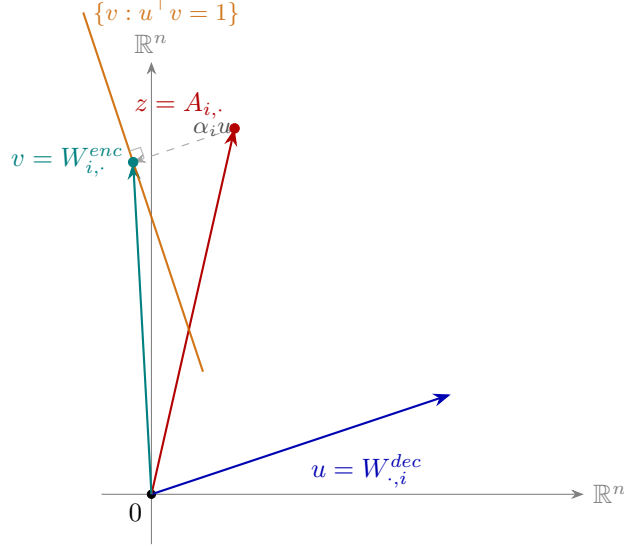


Figure 1: Geometric interpretation of aligned training for a single feature i . The free parameter $z = A_{i,·}$ is projected onto the affine hyperplane $\{v : u^\top v = 1\}$ (orange line) by adding a scalar multiple $\alpha_i u$ of the decoder direction $u = W_{·,i}^{dec}$. The result $v = W_{i,·}^{enc}$ satisfies the alignment constraint by construction. The small square marks the right angle between $\alpha_i u$ and the hyperplane.

Reparameterization. Let $A \in \mathbb{R}^{m \times n}$ be an unconstrained trainable matrix. Define each encoder row as the projection of $A_{i,·}$ onto the hyperplane $\{v : v \cdot W_{·,i}^{dec} = 1\}$:

$$W_{i,·}^{enc} := A_{i,·} + \alpha_i W_{·,i}^{dec}, \quad \alpha_i = \frac{1 - A_{i,·} \cdot W_{·,i}^{dec}}{\|W_{·,i}^{dec}\|^2}. \quad (2)$$

One verifies immediately that $W_{i,·}^{enc} \cdot W_{·,i}^{dec} = 1$ for every i . The biases $\mathbf{b}^{enc}$, $\mathbf{b}^{dec}$ and the decoder W^{dec} remain unrestricted trainable parameters; standard gradient descent is applied to $(A, \mathbf{b}^{enc}, \mathbf{b}^{dec}, W^{dec})$. Figure 1 illustrates the geometry.

Derivation. The constraint $u^\top v = 1$ (with $u = W_{·,i}^{dec}$, $v = W_{i,·}^{enc}$) defines an affine hyperplane with normal u . Any point on this hyperplane can be written as $v = z + \alpha u$ for a free parameter $z \in \mathbb{R}^n$. Substituting into the constraint:

$$u^\top (z + \alpha u) = 1 \implies \alpha = \frac{1 - u^\top z}{\|u\|^2}.$$

Equivalently, every solution decomposes as

$$v = \frac{u}{\|u\|^2} + \underbrace{\left(I - \frac{uu^\top}{\|u\|^2} \right)}_{\text{proj onto } u^\perp} z, \quad (3)$$

a fixed particular solution $u/\|u\|^2$ plus a component lying in the null space of $u^\top$ (the tangent space of the hyperplane). Equation (2) is exactly this formula. An alternative derivation via the Moore-Penrose pseudoinverse is given in Appendix D.

Compression for free. Since equation (2) constrains each encoder row to lie on an affine hyperplane of dimension $n - 1$, the last element of each row of A can be fixed to zero and left untrained. The aligned SAE therefore has slightly *fewer* parameters than the standard SAE while expressing the same space of reachable weights.

Connection to inductive biases. This approach is analogous to convolutional networks: a convolutional layer is a linear layer constrained by local receptive fields and weight sharing [11, 20]. In principle a linear layer could learn convolution, but the inductive bias makes training more effective. Here, aligned training constrains the encoder to a manifold that concentrates probability on the good basin of the optimization landscape.

4 Experiments

4.1 Setup

We train ReLU, TopK [13], and BatchTopK [6] sparse autoencoders on 50 million tokens from The Pile [12], evaluated on OpenWebText. The primary evaluation models are layer 8 of Pythia 160M and layer 12 of Gemma 2 2B [31], following the SAEBench protocol [19]. We train at dictionary sizes 4K, 16K, and 65K; for ReLU SAEs we sweep four sparsity penalties λ . All comparisons are across matched sparsity levels measured by L_0 . Full model and hyperparameter details are in Appendix E.

4.2 Reconstruction Metrics

Metrics. Reconstruction is evaluated by explained variance and recovered cross-entropy loss at varying sparsity levels. Explained variance is

$$1 - \frac{\frac{1}{K} \sum_k \|\mathbf{x}_k - \hat{\mathbf{x}}_k\|^2}{\frac{1}{K} \sum_k \|\mathbf{x}_k - \boldsymbol{\mu}\|^2},$$

where $\boldsymbol{\mu}$ is the mean activation. Recovered cross-entropy is

$$\frac{H^* - H_0}{H_{orig} - H_0},$$

with H_{orig} the model’s next-token cross-entropy, H^* the cross-entropy when activations are replaced by SAE reconstructions, and H_0 the cross-entropy when activations are zeroed.

Results. Aligned training achieves Pareto improvements over standard training across both models, all three dictionary sizes, and all four sparsity levels (Figure 2; see also Figures 15–17 in Appendix G).

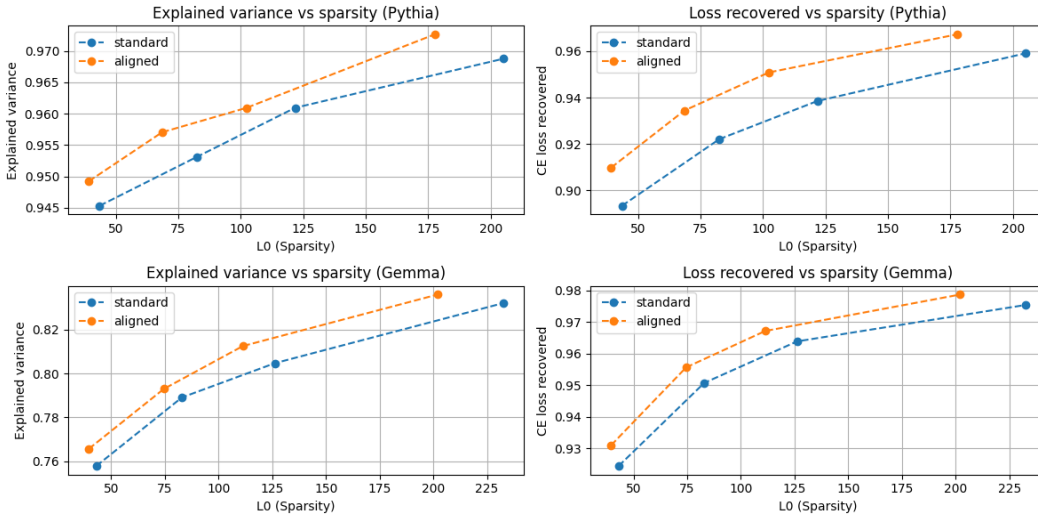


Figure 2: Aligned training improves recovered cross-entropy across different sparsity levels. Dictionary size 4096, layer 8 of Pythia 160M and layer 12 of Gemma 2 2B.

We further validate this at scale: Appendix H scales training to 500M tokens from The Pile and compares against the SAEBench state-of-the-art checkpoints, where aligned training continues to outperform on reconstruction metrics.

TopK and BatchTopK. Aligned training extends directly to TopK and BatchTopK architectures, where sparsity is enforced by the top- k activation rather than an L^1 penalty. Figure 3 shows improvements in the low-sparsity regime for the 65K Gemma dictionary. Results are consistent across dictionary sizes.

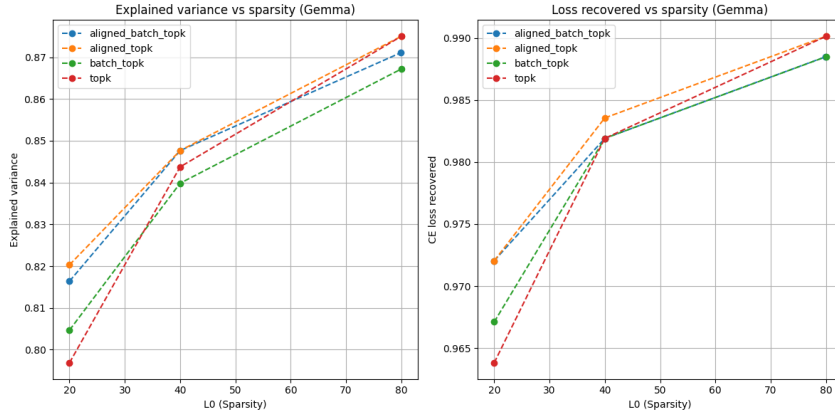


Figure 3: Aligned training improves TopK and BatchTopK autoencoders in the low-sparsity regime. Dictionary size 65K, layer 12 of Gemma 2 2B.

4.3 Dead Features

Dead features — features that never activate — are a persistent training pathology. Standard approaches address them after the fact: resampling [5], ghost gradients [16], and auxiliary losses [13] all add hyperparameters or non-differentiable operations. Standard ReLU SAEs exhibit approximately 20% dead features at dictionary size 4K.

Aligned training eliminates dead features structurally rather than resurrect them after they die. By constraining the optimization manifold, it prevents features from entering the zero-activation regime in the first place. Figure 4 shows that aligned training reduces dead features to near zero across both Pythia and Gemma, at no additional cost. The same effect holds for TopK and BatchTopK (Figure 5) and at 500M-token scale (Appendix H).

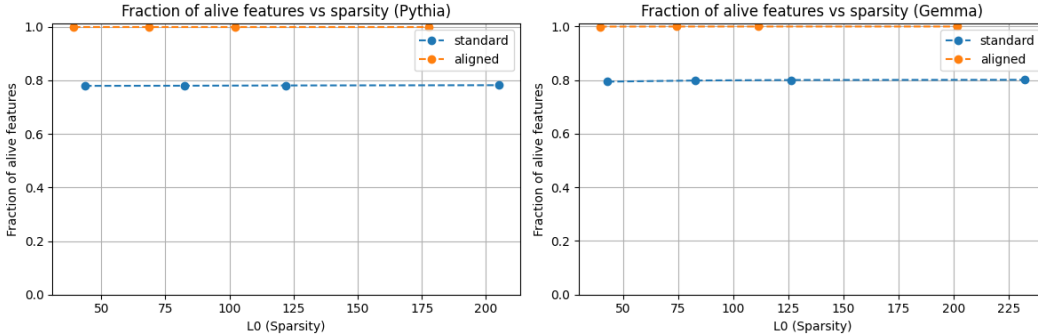


Figure 4: Aligned training reduces dead features to near zero without resampling or auxiliary losses. Dictionary size 4096, layer 8 of Pythia 160M and layer 12 of Gemma 2 2B.

4.4 Stability Across Seeds

Motivation. A natural question is whether two SAEs trained independently on the same data, same architecture, and same hyperparameters but different random seeds learn similar dictionaries. Recent work [23, 30] shows the answer is often no: standard SAEs exhibit substantial instability, with independently trained models converging to qualitatively different dictionaries. This undermines reproducibility in mechanistic interpretability.

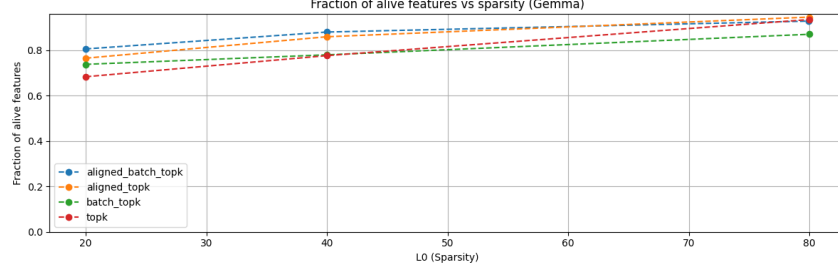


Figure 5: The dead-feature reduction extends to TopK and BatchTopK. Dictionary size 65K, layer 12 of Gemma 2 2B.

Metric. We measure stability as the mean min cosine similarity (MMCS) between the decoder dictionaries of two independently trained SAEs: for each feature in the first SAE we find its nearest neighbor in the second and average the cosine similarities. A score of 1 indicates perfect agreement; lower scores indicate divergent dictionaries.

Why alignment helps. Standard SAEs have two sources of instability beyond the inherent permutation symmetry: (i) the homogeneous rescaling degeneracy of Section 2.1, and (ii) the independent encoder and decoder, which introduce rotational degrees of freedom in the optimization landscape. Aligned training removes (ii) directly: the constraint $W_{i,\cdot}^{enc} \cdot W_{\cdot,i}^{dec} = 1$ couples each encoder row to its decoder column, shrinking the manifold of reachable weights and biasing optimization toward a more consistent basin.

Results. Figure 6 shows MMCS as a function of training progress for standard and aligned TopK SAEs (Gemma 2 2B, layer 12, dictionary size 65K, $k = 20$). Aligned TopK achieves consistently lower cosine distance throughout training, and the gap opens early, demonstrating the effect is not merely an initialization artifact.

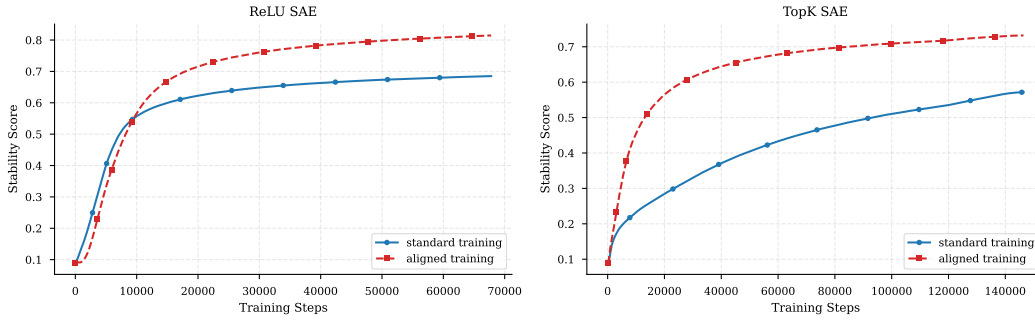


Figure 6: Aligned training significantly improves cross-seed stability for both ReLU and TopK autoencoders. Dictionary size 65K, layer 12 of Gemma 2 2B.

4.5 Combined with p-Annealing

The L^1 sparsity penalty in the standard SAE is a convex relaxation of the intractable L^0 norm, and is known to induce *feature shrinkage*: active features are systematically underestimated in magnitude because the penalty penalizes their activation values directly [1]. p-Annealing [18] addresses this by replacing the L^1 penalty with a smooth L^p quasnorm ($0 < p \leq 1$) and annealing p toward zero during training. Since L^p for $p < 1$ is nonconvex but smooth, it provides a closer approximation to L^0 while remaining differentiable, and at inference the resulting model is architecturally identical to a standard ReLU SAE. Feature shrinkage and the encoder-decoder misalignment we identify are orthogonal failure modes, so the two methods compose without interference.

Figure 7 and Figure 8 compare p-annealing alone against aligned training combined with p-annealing on Pythia 160M (layer 8), dictionary size 4096. On dead features, the combination reduces the dead

fraction to near zero across all sparsity levels, whereas p-annealing alone plateaus at roughly 73% alive features (Figure 8). On reconstruction, the combination achieves higher explained variance across most sparsity levels, with p-annealing alone showing a marginal advantage on CE loss recovered only at the highest sparsity levels (Figure 7). Overall, aligned training provides a consistent improvement over p-annealing alone, particularly on feature utilization.

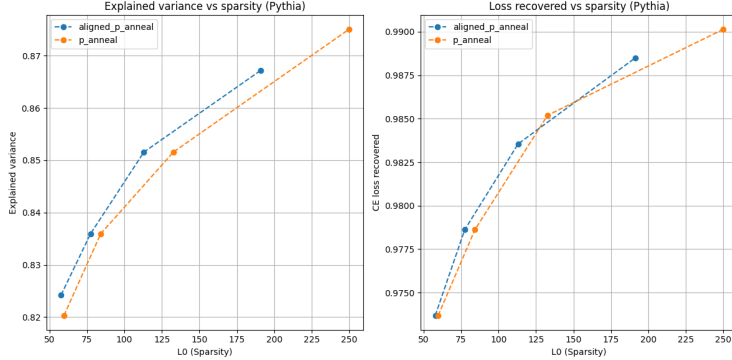


Figure 7: Reconstruction metrics for Pythia 160M (layer 8), dictionary size 4096, 3 random seeds.

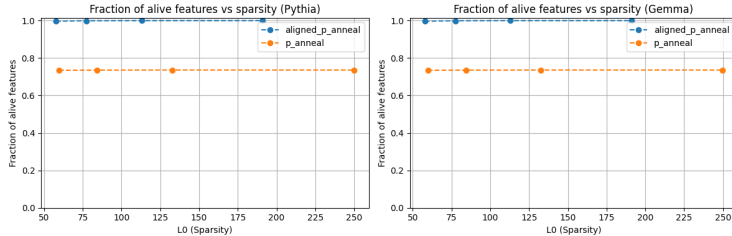


Figure 8: Alive-feature fraction for Pythia 160M (layer 8), dictionary size 4096.

4.6 Spurious Correlation Removal

Reconstruction quality is a proxy metric: it measures how faithfully the SAE reproduces activations, but not whether the learned features are useful for downstream interpretability tasks. To assess the latter, we evaluate on the Spurious Correlation Removal (SCR) benchmark from SAEBench [19], a feature disentanglement task adapted from the SHIFT method [22, 17].

In SCR, a classifier is trained to predict a target concept (e.g., profession) from SAE latents, where the training data contains a spurious correlation with a confounding attribute (e.g., gender). SAE latents identified as encoding the spurious signal are zero-ablated, and the resulting classifier accuracy on a balanced held-out set measures how cleanly the SAE has disentangled the two concepts. A higher SCR score indicates that the SAE has isolated the spurious attribute into dedicated latents, making it surgically removable without collateral damage to the target concept.

Figure 9 shows SCR scores at dictionary size 65K for Pythia 160M and Gemma 2 2B. On Pythia, aligned training outperforms the standard baseline consistently across all sparsity levels. On Gemma the picture is more nuanced: standard training has a marginal advantage at very low sparsity ($L_0 \leq 110$), after which aligned training overtakes it and the gap widens steadily with increasing L_0 . This pattern suggests that the benefits of alignment for feature disentanglement become more pronounced as the SAE is forced to be more selective about which features it activates.

5 Limitations

Limitations. Our empirical evaluation focuses on two model families (Pythia 160M and Gemma 2 2B) and a fixed set of SAE architectures. Whether the benefits of aligned training persist at larger

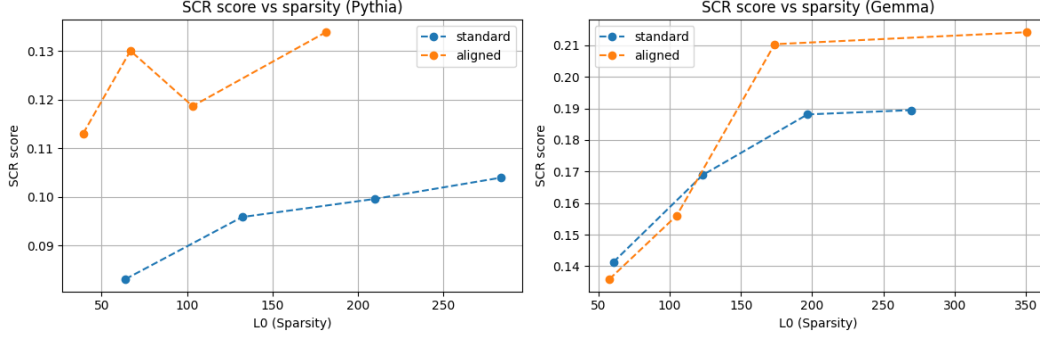


Figure 9: SCR metric from SAE Bench, dictionary size 65K, Pythia 160M and Gemma 2 2B.

model scales or under substantially different training regimes remains an open question for future work.

6 Conclusion

Despite the popularity and success of SAE in understanding the activations in DNNs, high dimensional features extracted from SAE exhibit critical challenges such as instability and dead features. To improve feature quality in a wide range of models without incurring additional training or data, we introduce the aligned training.

The alignment score is the inner product between each SAE feature’s encoder row and decoder column, which are shown to follow a bimodal distribution in standard SAEs and strongly correlates with existing quality proxies. The proposed aligned training enforces the alignment score to be 1 for every feature via a parameter-free reparameterization: the encoder is defined as the projection of a free parameter matrix A onto the affine hyperplane $\{v : v \cdot W_{:,i}^{dec} = 1\}$.

This single geometric constraint, with no additional hyperparameters, simultaneously produces three improvements: Pareto improvements in reconstruction quality, elimination of dead features, and significantly more stable dictionaries across seeds. These improvements are observed across different models, dictionary sizes, sparsity levels, and SAE architectures. Training budgets of 50M and 500M tokens resulted in similar improvements. Furthermore, we have shown that other training techniques for SAE such as p-Annealing can be used together with the aligned training.

The stability result directly addresses the feature consistency concerns raised by Paulo and Belrose [23] and Song et al. [30] as a barrier to reliable mechanistic interpretability. Paulo and Belrose [23] show that in large SAEs, as few as 30% of features are shared across independent training runs, and that this instability is particularly pronounced for TopK architectures. Song et al. [30] argue that consistency should be treated as a primary design objective rather than an afterthought. Aligned training addresses this directly: by coupling each encoder row to its decoder column through the constraint $W_{i,:}^{enc} \cdot W_{:,i}^{dec} = 1$, it removes a rotational degree of freedom that allows different seeds to converge to geometrically distinct but equally valid dictionaries. As shown in Figure 6, the MMCS gap between aligned and standard TopK SAEs opens early in training and widens throughout, indicating the improvement is structural rather than an artifact of initialization.

References

- [1] Lee Sharkey Benjamin Wright. Addressing feature suppression in saes. *AI Alignment Forum*, 2024. URL <https://www.alignmentforum.org/posts/3JuSjTZyMzaSeTxKk/addressing-feature-suppression-in-saes>.
- [2] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.

- [3] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. Language models can explain neurons in language models, 2023. URL <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>.
- [4] Joseph Bloom, Curt Tigges, Anthony Duong, and David Chanin. Saelens. <https://github.com/jbloomAus/SAELens>, 2024.
- [5] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [6] Bart Bussmann, Patrick Leask, and Neel Nanda. Batchtopk sparse autoencoders. In *NeurIPS 2024 Workshop on Scientific Methods for Understanding Deep Learning*, 2024. URL <https://openreview.net/forum?id=d4dp0CqyBL>.
- [7] Tom Conerly, Adly Templeton, Trenton Bricken, Jonathan Marcus, and Tom Henighan. Update on dictionary learning improvements. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/april-update/index.html#training-saes>.
- [8] Hoagy Cunningham. Autointerpretation finds sparse coding beats alternatives. *AI Alignment Forum*, 2023. URL <https://www.alignmentforum.org/posts/ursraZGcpfMjCXtnn/autointerpretation-finds-sparse-coding-beats-alternatives>.
- [9] Hoagy Cunningham and Logan Riggs. [replication] conjecture’s sparse coding in small transformers. *Less Wrong*, 2023. URL <https://www.lesswrong.com/posts/vBcsAw4rvLsri3JAj/replication-conjecture-s-sparse-coding-in-small-transformers>.
- [10] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.
- [11] Kunihiko Fukushima. Neocognitron: A hierarchical neural network capable of visual pattern recognition. *Neural Networks*, 1(2):119–130, 1988. ISSN 0893-6080. doi: [https://doi.org/10.1016/0893-6080\(88\)90014-7](https://doi.org/10.1016/0893-6080(88)90014-7). URL <https://www.sciencedirect.com/science/article/pii/0893608088900147>.
- [12] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- [13] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tcsZt9ZNKD>.
- [14] Robert Huben. [research update] sparse autoencoder features are bimodal. *From AI to ZI*, 2023. URL <https://aizi.substack.com/p/research-update-sparse-autoencoder>.
- [15] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2023.
- [16] Adam Jermyn and Adly Templeton. Ghost grads: An improvement on resampling. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/jan-update/index.html#dict-learning-resampling>.

- [17] Adam Karvonen, Can Rager, Samuel Marks, and Neel Nanda. Evaluating sparse autoencoders on targeted concept erasure tasks, 2024. URL <https://arxiv.org/abs/2411.18895>.
- [18] Adam Karvonen, Benjamin Wright, Can Rager, Rico Angell, Jannik Brinkmann, Logan Riggs Smith, Claudio Mayrink Verdun, David Bau, and Samuel Marks. Measuring progress in dictionary learning for language model interpretability with board game models. In *ICML 2024 Workshop on Mechanistic Interpretability*, 2024. URL <https://openreview.net/forum?id=qzsDKwGJyB>.
- [19] Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Demian Till, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. Saebench: A comprehensive benchmark for sparse autoencoders in language model interpretability, 2025. URL <https://arxiv.org/abs/2503.09532>.
- [20] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- [21] Luke Marks, Alasdair Paren, David Krueger, and Fazl Barez. Enhancing neural network interpretability with feature-aligned sparse autoencoders, 2024. URL <https://arxiv.org/abs/2411.01220>.
- [22] Samuel Marks, Can Rager, Eric J Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=I4e82CIDxv>.
- [23] Gonalo Paulo and Nora Belrose. Sparse autoencoders trained on the same data learn different features. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=EjInprGpk9>.
- [24] Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah, and Neel Nanda. Improving sparse decomposition of language model activations with gated sparse autoencoders. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 775–818. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/01772a8b0420baec00c4d59fe2fbace6-Paper-Conference.pdf.
- [25] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, Janos Kramar, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumpReLU sparse autoencoders, 2025. URL <https://openreview.net/forum?id=mMPaQzgZAN>.
- [26] Logan Riggs. (tentatively) found 600+ monosemantic features in a small lm using sparse autoencoders. *AI Alignment Forum*, 2023. URL <https://www.alignmentforum.org/posts/wqRqb7h6ZC48iDgfk/tentatively-found-600-monosemantic-features-in-a-small-lm>.
- [27] Alex Rogozhnikov. Einops: Clear and reliable tensor manipulations with einstein-like notation. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=oapKSVM2bcj>.
- [28] Adam Karvonen Samuel Marks and Aaron Mueller. dictionary_learning, 2024. URL https://github.com/saprmarks/dictionary_learning.
- [29] Lee Sharkey, Dan Braun, and Beren Millidge. Taking features out of superposition with sparse autoencoders. *Alignment Forum*, 2023. URL <https://www.alignmentforum.org/posts/z6QQJbtpkEAX3AoJJ/interim-research-report-taking-features-out-of-superposition>.

- [30] Xiangchen Song, Aashiq Muhamed, Yujia Zheng, Lingjing Kong, Zeyu Tang, Mona T. Diab, Virginia Smith, and Kun Zhang. Position: Mechanistic interpretability should prioritize feature consistency in SAEs. In *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025. URL <https://openreview.net/forum?id=d9ACURK6bI>.
- [31] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, Anton Tsitsulin, Nino Vieillard, Piotr Stanczyk, Sertan Girgin, Nikola Momchev, Matt Hoffman, Shantanu Thakoor, Jean-Bastien Grill, Behnam Neyshabur, Olivier Bachem, Alanna Walton, Aliaksei Severyn, Alicia Parrish, Aliya Ahmad, Allen Hutchison, Alvin Abdagic, Amanda Carl, Amy Shen, Andy Brock, Andy Coenen, Anthony Laforge, Antonia Paterson, Ben Bastian, Bilal Piot, Bo Wu, Brandon Royal, Charlie Chen, Chintu Kumar, Chris Perry, Chris Welty, Christopher A. Choquette-Choo, Danila Sinopalnikov, David Weinberger, Dimple Vijaykumar, Dominika Rogozińska, Dustin Herbison, Elisa Bandy, Emma Wang, Eric Noland, Erica Moreira, Evan Senter, Evgenii Eltyshev, Francesco Visin, Gabriel Rasskin, Gary Wei, Glenn Cameron, Gus Martins, Hadi Hashemi, Hanna Klimczak-Plucińska, Harleen Batra, Harsh Dhand, Ivan Nardini, Jacinda Mein, Jack Zhou, James Svensson, Jeff Stanway, Jetha Chan, Jin Peng Zhou, Joana Carrasqueira, Joana Iljazi, Jocelyn Becker, Joe Fernandez, Joost van Amersfoort, Josh Gordon, Josh Lipschultz, Josh Newlan, Ju yeong Ji, Kareem Mohamed, Kartikeya Badola, Kat Black, Katie Millican, Keelin McDonell, Kelvin Nguyen, Kiranbir Sodhia, Kish Greene, Lars Lowe Sjoesund, Lauren Usui, Laurent Sifre, Lena Heuermann, Leticia Lago, Lilly McNealus, Livio Baldini Soares, Logan Kilpatrick, Lucas Dixon, Luciano Martins, Machel Reid, Manvinder Singh, Mark Iverson, Martin Görner, Mat Velloso, Mateo Wirth, Matt Davidow, Matt Miller, Matthew Rahtz, Matthew Watson, Meg Risdal, Mehran Kazemi, Michael Moynihan, Ming Zhang, Minsuk Kahng, Minwoo Park, Mofi Rahman, Mohit Khatwani, Natalie Dao, Nenshad Bardoliwalla, Nesh Devanathan, Neta Dumai, Nilay Chauhan, Oscar Wahltinez, Pankil Botarda, Parker Barnes, Paul Barham, Paul Michel, Pengchong Jin, Petko Georgiev, Phil Culliton, Pradeep Kuppala, Ramona Comanescu, Ramona Merhej, Reena Jana, Reza Ardeshtir Rokni, Rishabh Agarwal, Ryan Mullins, Samaneh Saadat, Sara Mc Carthy, Sarah Cogan, Sarah Perrin, Sébastien M. R. Arnold, Sebastian Krause, Shengyang Dai, Shruti Garg, Shruti Sheth, Sue Ronstrom, Susan Chan, Timothy Jordan, Ting Yu, Tom Eccles, Tom Hennigan, Tomas Kocisky, Tulsee Doshi, Vihan Jain, Vikas Yadav, Vilobh Meshram, Vishal Dharmadhikari, Warren Barkley, Wei Wei, Wenming Ye, Woohyun Han, Woosuk Kwon, Xiang Xu, Zhe Shen, Zhitao Gong, Zichuan Wei, Victor Cotruta, Phoebe Kirk, Anand Rao, Minh Giang, Ludovic Peran, Tris Warkentin, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, D. Sculley, Jeanine Banks, Anca Dragan, Slav Petrov, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Sebastian Borgeaud, Noah Fiedel, Armand Joulin, Kathleen Kenealy, Robert Dadashi, and Alek Andreev. Gemma 2: Improving open language models at a practical size, 2024. URL <https://arxiv.org/abs/2408.00118>.
- [32] Zhengxuan Wu, Aryaman Arora, Atticus Geiger, Zheng Wang, Jing Huang, Dan Jurafsky, Christopher D Manning, and Christopher Potts. Axbench: Steering LLMs? even simple base-lines outperform sparse autoencoders. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=K2CckZjNy0>.

A Implementation Details

All SAEs are trained on 50 million tokens from The Pile, evaluated on OpenWebText, following the SAEBench protocol. The primary evaluation models are layer 8 of Pythia 160M (residual stream dimension 768, float32) and layer 12 of Gemma 2 2B (residual stream dimension 2304, bfloat16). All experiments are conducted on a single NVIDIA H100 80GB GPU. A single SAE training run completes in approximately 13 minutes on this hardware.

Training uses the Adam optimizer with learning rate 3×10^{-4} , a linear warmup over 1000 steps, sparsity penalty warmup over 5000 steps, and learning rate decay beginning at 80% of total training steps. The SAE batch size is 2048 for all models. Dictionary sizes are 4K, 16K, and 65K. For ReLU SAEs, we sweep four sparsity penalties $\lambda \in \{0.025, 0.035, 0.045, 0.07\}$.

B Code Implementation

For training our autoencoders, we utilized the code [28] and implemented the key transform (2) using the Python function below. For computing per-feature inner products we used the einops [27] library.

```
def get_the_encoder_matrix(
    dict_size: int,
    encoder_weights_orthogonal_part: Float[Tensor, "activation_dim-1 dict_size"],
    decoder_weights: Float[Tensor, "dict_size activation_dim"]
) -> Float[Tensor, "activation_dim dict_size"]:
    zeros = torch.zeros(1, dict_size).to(
        encoder_weights_orthogonal_part)
    appended = torch.concat([encoder_weights_orthogonal_part, zeros])
    inner_products = einops.einsum(
        decoder_weights, appended,
        "dict_size activation_dim, activation_dim dict_size -> dict_size"
    )
    decoder_norms_squared = decoder_weights.pow(2).sum(dim=1)
    reparametrized = appended + decoder_weights.T * (1 -
        inner_products) / decoder_norms_squared
    return reparametrized
```

C Alignment Score: Bimodality and Correlations

C.1 Alignment Scores Are Bimodal

We computed histograms of the alignment score across Pythia 70M, LLaMA 3 8B IT, and Gemma 2 2B, using standard pretrained SAEs loaded via the sae-lens framework [4]. The score distribution is consistently bimodal across all models and architectures (Figure 10). This phenomenon mirrors the bimodality of MCS reported by Huben [14], but the alignment score requires no additional training run. Specific SAE details are in Appendix E.

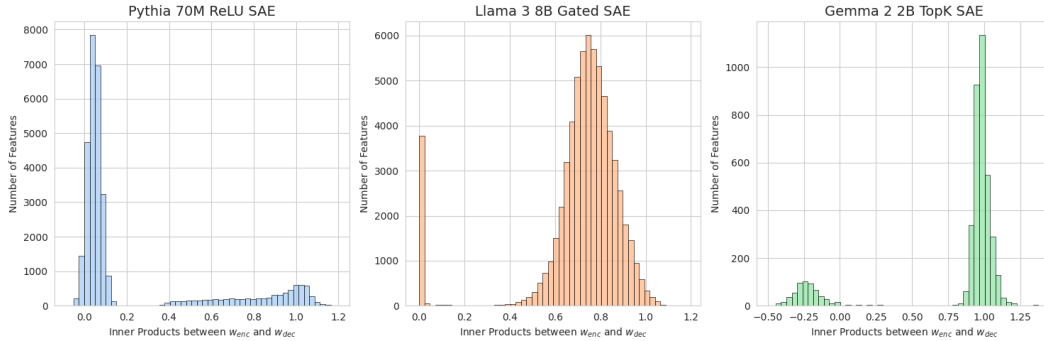


Figure 10: Bimodality of SAE alignment scores across different models and architectures.

C.2 Alignment Scores Are Highly Correlated with MCS

We compared alignment scores with MCS for an SAE trained on MLP activations from layer 2 of Pythia [2], using the SAEs from [26]. Figure 11 shows a Pearson correlation of 0.65; low-alignment features cluster away from the diagonal and are consistently missed by the larger interpreter model. The best features concentrate near $a_i = 1$, consistent with the toy model prediction (Section 3.1).

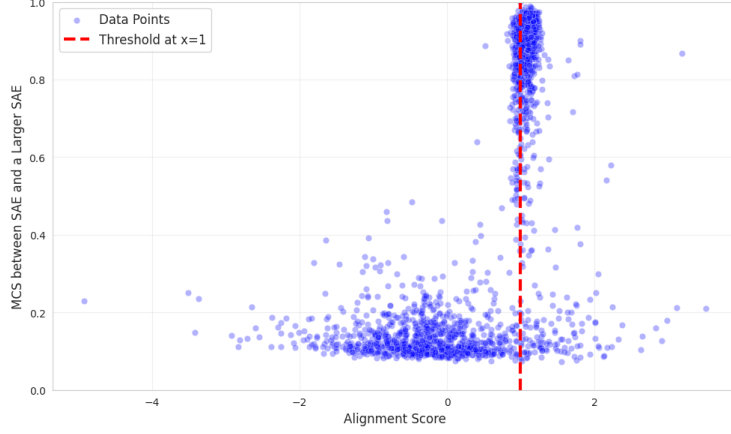


Figure 11: MCS vs. alignment score (Pearson $r = 0.65$). The red vertical line marks $a_i = 1$.

C.3 Alignment Scores Are Correlated with Autointerpretability

The alignment score is positively correlated with autointerpretability (Pearson $r = 0.32$; Figure 12), using the protocol from [3] with Gemma 3 27B IT as judge. Dead features (insufficient non-zero activations) are shown as black dots.

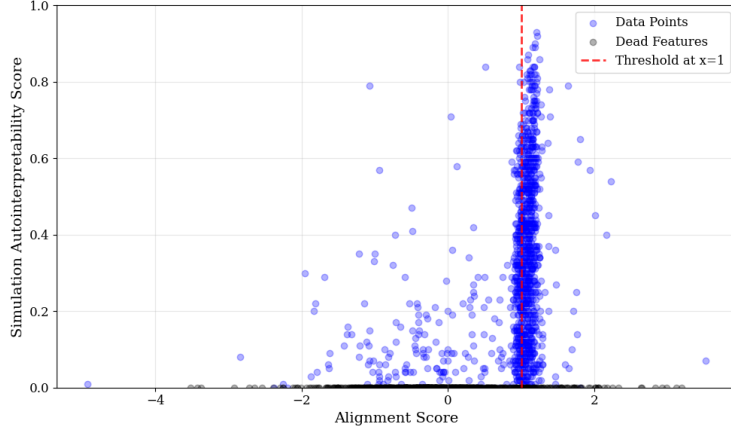


Figure 12: Autointerpretability vs. alignment score (Pearson $r = 0.32$). Red line marks $a_i = 1$.

D Alternative Derivation via Pseudoinverse

The inline derivation in Section 3.2 parameterizes the affine hyperplane $\{v : u^\top v = 1\}$ by shifting a free vector z along the normal u . The same result follows from the Moore-Penrose pseudoinverse. The general solution of $u^\top v = 1$ is

$$v = (u^\top)^+ + (I - (u^\top)^+ u^\top) z, \quad z \in \mathbb{R}^n,$$

where for a row vector $u^\top$ the pseudoinverse is $(u^\top)^+ = u / \|u\|^2$. Substituting yields equation (3) immediately, confirming that the two formulations are identical.

E Models and SAE Details

For loading pretrained SAEs we used the sae-lens framework [4]. SAEs used in Appendix C:

- Pythia: 70M, residual stream layer 3. Release: *pythia-70m-deduped-res-sm*, id: *blocks.3.hook_resid_post*.
- LLaMA: 3 8B IT, residual stream layer 25. Release: *llama-3-8b-it-res-jh*, id: *blocks.25.hook_resid_post*.
- Gemma: 2 2B, residual stream layer 12. Release: *sae_bench_gemma-2-2b_topk_width-2pow12_date-1109*, id: *blocks.12.hook_resid_post__trainer_0*.

For the MCS and autointerpretability plots we used the ReLU autoencoders from [26]. For the remaining experiments we used layer 8 of Pythia 160M and layer 12 of Gemma 2 2B, following SAEBench.

F Comparison to Weight-Tying

Weight tying [15] enforces $W^{enc} = (W^{dec})^T$, a strictly stronger constraint than aligned training. Experiments show that weight tying reduces dead features but compromises reconstruction quality (Figures 13–14). Aligned training achieves improvements on both metrics simultaneously.

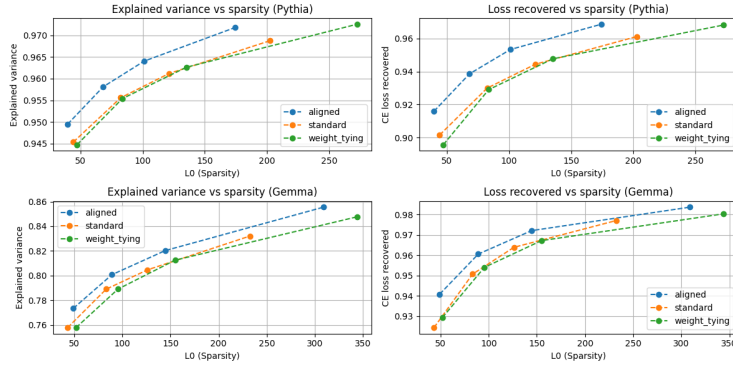


Figure 13: Reconstruction metrics for Pythia 160M (layer 8) and Gemma 2 2B (layer 12), dictionary size 16384. Weight tying underperforms on reconstruction.

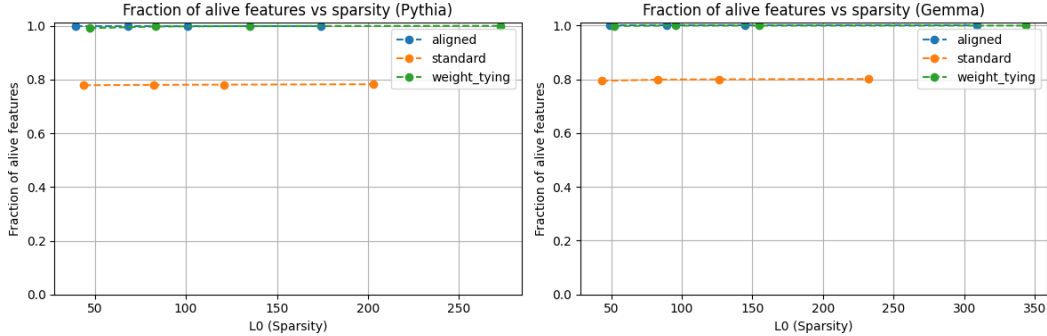


Figure 14: Weight tying reduces dead features but at the cost of reconstruction quality. Pythia 160M (layer 8) and Gemma 2 2B (layer 12), dictionary size 16384.

G Results for Different Dictionary Sizes

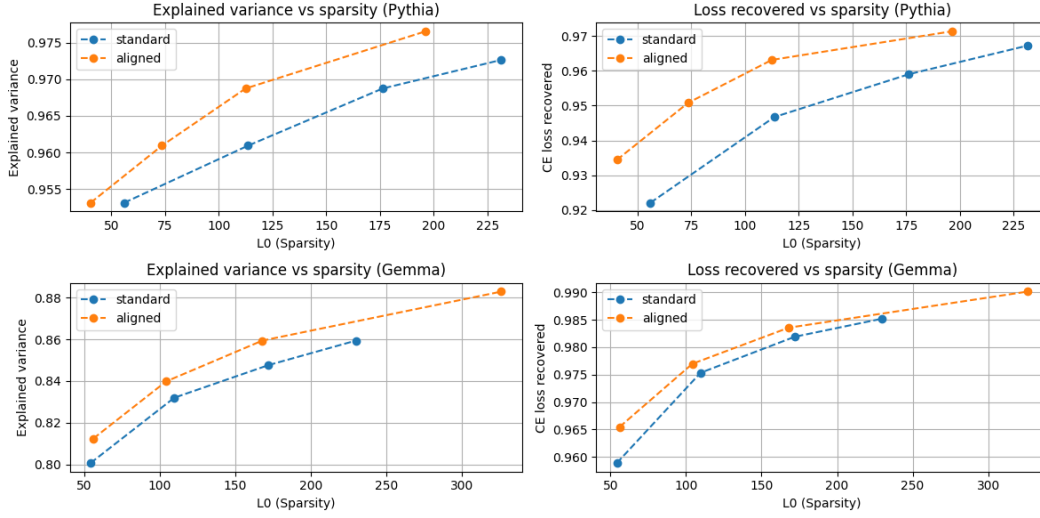


Figure 15: Reconstruction metrics, dictionary size 16384, Pythia 160M (layer 8) and Gemma 2 2B (layer 12).

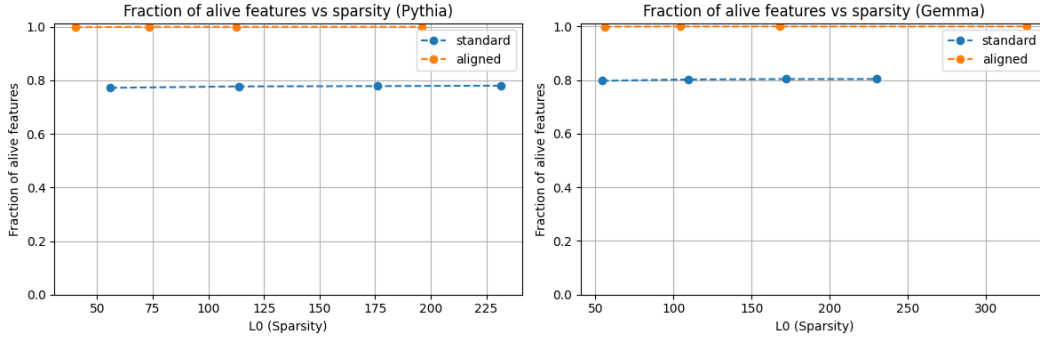


Figure 16: Alive-feature fraction, dictionary size 16384.

H Scaling to 500M Tokens and State-of-the-Art Comparison

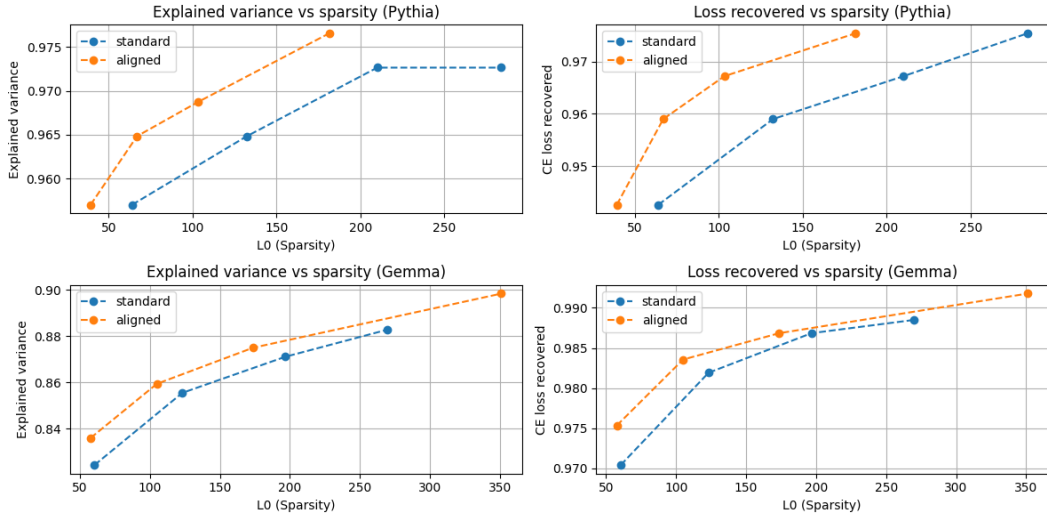


Figure 17: Reconstruction metrics, dictionary size 65K.

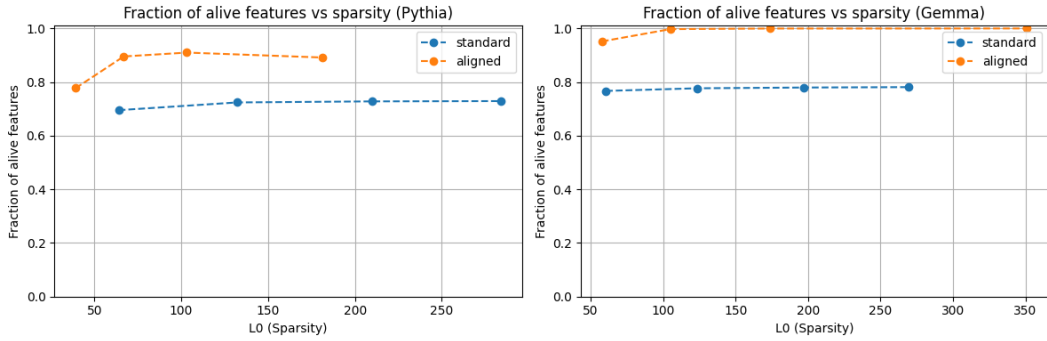


Figure 18: Alive-feature fraction, dictionary size 65K.

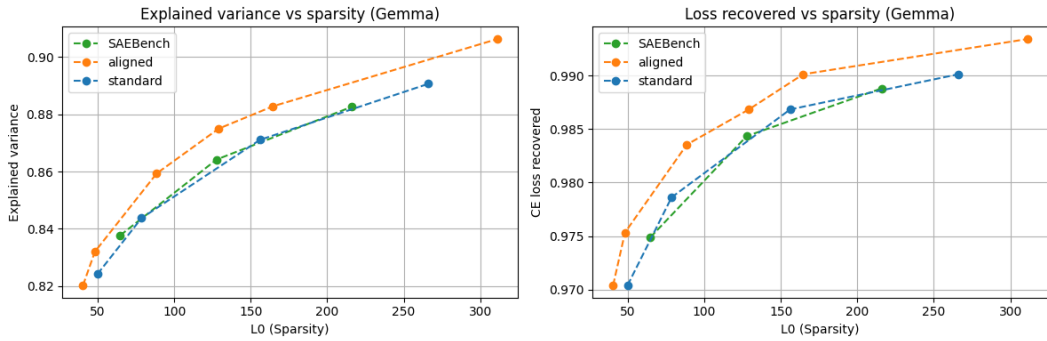


Figure 19: Reconstruction metrics at 500M tokens, dictionary size 65K, Gemma 2 2B (layer 12). Aligned training outperforms both our standard baseline and SAE-Bench state-of-the-art checkpoints.

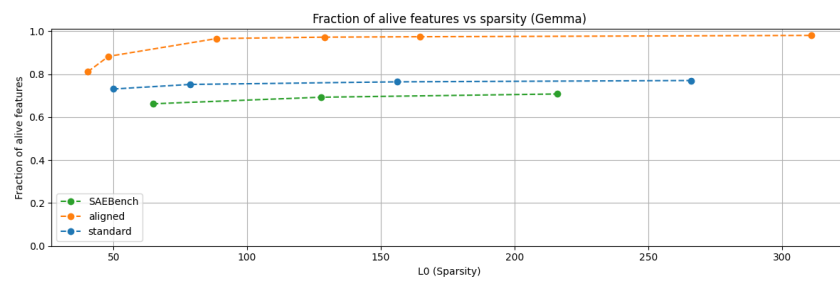


Figure 20: Alive-feature fraction at 500M tokens, dictionary size 65K, Gemma 2 2B (layer 12).